Distributed Mixture-of-Experts Inference Engine Across NVIDIA GB10 Swarms: Architectural Feasibility and System Blueprint

Literature and State-of-the-Art Review: Predictive Routing and Asynchronous Inference

Serving frontier-scale Mixture-of-Experts (MoE) architectures spanning hundreds of billions to multiple trillions of parameters introduces fundamental execution bottlenecks in conventional distributed runtimes[1]. Standard distributed systems rely on synchronous Tensor Parallelism (TP) or Expert Parallelism (EP) coordinated through collective communication libraries such as NCCL[3]. Under synchronous EP, the all-to-all communication primitive enforces strict barrier synchronization across all participant nodes; every token's routing decision must be fully evaluated at layer l before dispatching activations across the network to layer l's assigned experts[3]. Consequently, network transfer latency directly serializes the critical execution path, while dynamic load imbalances in token routing lead to straggler stalls that throttle entire GPU clusters[4].

Recent academic and open-source systems challenge this synchronous execution model by introducing predictive routing, peer-to-peer dataflow, and hierarchical tiered streaming. The landscape of these emerging runtimes reveals distinct trade-offs across execution paradigms, communication substrates, and memory hierarchies.

System Paradigm	Primary Execution Model	Communication Substrate	Memory Hierarchy & Staging	Scheduling & Synchronization
NCCL / Megatron-Core	Synchronous EP / TP	Two-Sided All-to-All Collectives	Monolithic Resident VRAM / HBM	Static Global Barrier Synchronization[3]
DeepEP / DeepEP V2	GPU-Initiated Asynchronous EP	One-Sided IBGDA over InfiniBand	Pre-allocated Global Buffer Pool	Hardware-Driven Doorbell Descriptors[8]

fabric-lib / FAST	Portable Point-to-Point RDMA	Unordered One-Sided WRITE_IMM	Dynamic Paged Memory Regions	Asynchronous ImmCounter Synchronization[4]
Director	Proactive Placement EP	Predictive Gating + RDMA Streams	Dynamic Live Weight Migration	Binary Search LP Reconfiguration Schedule[5]
Colibri / Lumabri	Just-In-Time Tiered Streaming	Peer-to-Peer TCP/IP Swarm	NVMe Storage $\rightarrow$ RAM $\rightarrow$ VRAM	Demand-Driven JIT Cache Scheduler[10]
Strata	Asynchronous Layer Overlap	GPU-Assisted Direct I/O	Host-Pinned RAM KV Streaming	Fine-Grained Pipelined Compute/Swap Overlap[13]
TensorFold	Bare-Metal Speculative Decoding	Direct CUDA / Metal C-ABI	Unified Memory Arena Allocator	Deterministic Speculative Draft Loop[15]

The Director serving framework formalizes online proactive expert placement to mitigate routing path uncertainty in distributed MoE serving[5]. In standard MoE architectures, the gating network at layer l depends strictly on the hidden representation produced by the experts at layer $l-1$, creating tight sequential coupling[5]. Director demonstrates that expert activation patterns can be predicted ahead of layer execution using either lightweight cascaded neural predictors or low-bit quantized model replicas, reducing end-to-end serving latency by 11% to 55% across models such as Mistral, DeepSeek, and Qwen[5]. The system decouples routing prediction from physical execution, transforming reactive communication schedules into proactive schedules that overlap network migration with upstream layer computation[5].
DeepEP pioneered high-throughput, low-latency GPU-initiated RDMA communication specifically tailored for MoE dispatch and combine phases[8]. DeepEP leverages InfiniBand GPUDirect Async (IBGDA) and NVSHMEM, enabling Streaming Multiprocessors (SMs) to write Work Queue Elements (WQEs) directly to ConnectX network interface cards (NICs) without CPU proxy involvement[3]. However, DeepEP enforces vendor-specific couplings and collective-style buffer sizing[4].
To overcome these constraints, fabric-lib introduces an asynchronous point-to-point RDMA transport architecture built directly on libibverbs[4]. Fabric-lib introduces the ImmCounter primitive, which relies on one-sided RDMA WRITE with immediate data to track message

completions without enforcing global ordering, enabling state-of-the-art token dispatch and combine latency on ConnectX-7 adapters while eliminating synchronization barriers[4].
On the storage and memory hierarchy front, the Colibri and Lumabri systems demonstrate that trillion-parameter MoE inference is achievable on memory-constrained hardware by abandoning the requirement that all model weights remain resident in fast memory[1]. Colibri operates as a just-in-time compiler for model weights, maintaining dense transformer backbones in memory while streaming sparsely routed experts from NVMe storage over high-speed I/O channels[11]. Lumabri extends this concept into a peer-to-peer household swarm, sharding expert storage and execution across disparate LAN nodes using a pure-C networking daemon[10].
Concurrently, Strata demonstrates that GPU-assisted I/O and custom low-level assembly kernels can pipeline KV cache swapping and weight loading alongside matrix compute, maximizing Tensor Core operational intensity and reducing Time-To-First-Token (TTFT) by up to 5× compared to standard runtimes[13].
TensorFold introduces bare-metal execution paradigms written in Zig, bypassing the Python runtime entirely to achieve deterministic, zero-overhead kernel dispatch and speculative lookahead decoding[15].
Synthesizing these distinct architectures onto a swarm of edge-scale NVIDIA GB10 nodes requires unifying TensorFold's bare-metal Zig/C scheduling loop, Lumabri's peer-to-peer memory management, and Strata's low-level Blackwell kernel primitives into an asynchronous, streaming expert-parallel engine.

Speculative Lookahead Routing and Predictive Gating Architecture

Mathematical Formulation of Lookahead Routing

In a standard Mixture-of-Experts transformer with L layers and E experts per layer, the token representation $x_t^{(l)} \in \mathbb{R}^{d_{model}}$ at sequence index t and layer l is routed according to gating logits produced by a linear projection $W_g^{(l)} \in \mathbb{R}^{E \times d_{model}}$ [2]:

$$h_t^{(l)} = W_g^{(l)} x_t^{(l)}$$

The routing distribution and the selection of the top-k active experts $\mathcal{E}_t^{(l)}$ are governed by:

$$S_t^{(l)} = \mathrm{Softmax}\left(\mathrm{TopK}\left(h_t^{(l)}, k\right)\right), \quad \mathcal{E}_t^{(l)} = \mathrm{argTopK}\left(h_t^{(l)}, k\right)$$

Because $x_t^{(l)}$ is an explicit function of the combined expert outputs from layer $l-1$, the routing set $\mathcal{E}_t^{(l)}$ cannot be resolved in a standard pipeline until layer $l-1$ finishes

execution[5]. To break this serialization, the proposed engine implements a speculative lookahead auxiliary network, denoted as $\mathcal{P}_\Theta$, which predicts the expert activation sets for layers $l+1, \ldots, l+H$ up to a lookahead horizon $H \in \{2, 3\}$.

The auxiliary lookahead router evaluates an input representation extracted at layer l, projecting forward across the horizon:

$$\hat{h}_t^{(l+m)} = \mathcal{P}_\Theta^{(m)}\left(x_t^{(l)}, \hat{\mathcal{E}}_t^{(l+1)}, \ldots, \hat{\mathcal{E}}_t^{(l+m-1)}\right), \quad \forall m \in [1, H]$$

Following the cascaded prediction formulation established in Director, the lookahead network is partitioned into lightweight stages[5]. The initial stage predicts layer $l+$ routing directly from $x_*^{(l)}$:

$$\hat{h}_t^{(l+1)} = W_{p,1}^{(l)} x_t^{(l)} + b_{p,1}^{(l)}$$

Subsequent prediction stages augment the intermediate hidden representation with the predicted routing distribution of preceding intermediate layers via learned embedding projections:

$$\hat{h}_t^{(l+m)} = W_{p,m}^{(l)}\left[x_t^{(l)} \parallel \sum_{e \in \hat{\mathcal{E}}_t^{(l+m-1)}} W_{emb}^{(l+m-1)}(e)\right] + b_{p,m}^{(l)}$$

where $\parallel$ denotes vector concatenation, and $W_{emb} \in \mathbb{R}^{E \times d_{proj}}$ embeds the discrete predicted expert identities into a continuous latent space. The predicted candidate expert set for layer $l+m$ is then selected as:

$$\hat{\mathcal{E}}_t^{(l+m)} = \text{argTopK}\left(\hat{h}_t^{(l+m)}, k + \Delta k\right)$$

where $\Delta k \geq 0$ represents an over-provisioning factor configured to capture boundary mispredictions[5].

The architectural flow of speculative lookahead routing executes across two parallel execution streams. The primary compute stream on the GPU evaluates the current layer's self-attention and matrix multiplications. Simultaneously, the speculative router stream on a dedicated CPU or auxiliary GPU warp evaluates the cascaded predictor. Stage 1 predicts the routing for layer $l+$, immediately initiating RDMA buffer pre-negotiation and memory key exchanges. Stage 2 projects forward to layer $l+2$, dispatching asynchronous NVMe direct I/O read requests through io_uring for any required experts not currently resident in memory. Stage 3 projects forward to layer $l+3$, pre-allocating descriptors within Tensor Memory (TMEM) and L2 cache

lines. Because these lookahead projections occur concurrently with upstream layer computation, the system creates an overlapping execution window that absorbs downstream communication and I/O latencies.

Compute Overhead Versus Network and Storage Latency Savings

To justify speculative routing, the computational cost of evaluating $\mathcal{P}_\Theta$ must remain negligible compared to the execution time of the primary transformer layers, while yielding sufficient lead time to hide ConnectX-7 RDMA all-to-all transfers and PCIe Gen 5 NVMe weight fetches[5]. Consider an archetype frontier MoE architecture (representative of GLM-5.2 and DeepSeek-V3 configurations) characterized by model dimension $d_{model} = 7168$, intermediate expert dimension $d_{ffn} = 2048$, $E = 256$ total experts, and $k = 8$ active experts per token[2]. Each expert utilizes a SwiGLU gated feed-forward network comprising three weight matrices ($W_{gate}, W_{up}, W_{down}$), yielding a parameter count per expert:

$$P_{expert} = 3 \times d_{model} \times d_{ffn} = 3 \times 7168 \times 2048 = 44,040,192 \text{ parameters}$$

The floating-point operations (FLOPs) required to execute one full transformer layer per token (combining self-attention projections and $k = 8$ expert FFN evaluations) total:

$$\text{FLOPs}_{layer} \approx 2 \times 4 \times d_{model}^2 + 2 \times k \times P_{expert} \approx 8 \times (7168)^2 + 16 \times 44.04 \times 10^6 \approx 1.115 \times 10^9 \text{ FL}$$

The cascaded lookahead router $\mathcal{P}_\Theta$ operates with an internal projection dimension $d_{proj} = 512$. For a lookahead horizon $H = 2$, the auxiliary predictor executes two consecutive linear projections per token:

$$\text{FLOPs}_{predictor} = 2 \times \sum_{m=1}^{2} (2 \times d_{model} \times d_{proj} + 2 \times d_{proj} \times E)$$

$$\text{FLOPs}_{predictor} = 4 \times (7168 \times 512 + 512 \times 256) = 4 \times (3,670,016 + 131,072) \approx 1.52 \times 10^7 \text{ FLOPs}$$

The computational overhead relative to the layer computation is:

$$\text{Overhead}_{compute} = \frac{1.52 \times 10^7}{1.115 \times 10^9} \approx 1.36\%$$

On the NVIDIA GB10 Superchip, whose Blackwell GPU delivers 475.4 TFLOPS of dense FP4 tensor compute, computing 1.52×10^7 FLOPs takes approximately $0.032\ \mu\text{s}$ per token[28]. For a distributed batch of 4,096 tokens, the predictor consumes approximately $131\ \mu\text{s}$ of

GPU compute time.
In exchange for this 1.36% compute overhead, predicting expert routing 2 layers ahead ($H = 2$) provides an execution window of approximately two complete transformer layers ($2 \times t_{layer} \approx 300\ \mu s$ to $1.2\ \text{ms}$, depending on batch size)[5]. This lookahead window enables zero-latency network dispatch, where RDMA queue pairs can be armed and remote memory handles negotiated before layer l completes, allowing token activations to be streamed over RoCE v2 the microsecond they exit the self-attention projection[4]. Furthermore, it facilitates asynchronous NVMe staging, whereby non-resident long-tail experts predicted for layer $l + 2$ can be scheduled for direct memory access (DMA) transfer via io_uring hundreds of microseconds before their execution kernels are scheduled, hiding PCIe transfer latency behind active compute[11].

Bare-Metal Zero-Copy RDMA Swarm Topology and All-to-All Dispatch

Bare-Metal RDMA Architecture in Zig/C

In an expert-parallel swarm, "weights stay put while activations move." Achieving wire-speed token transfer across GB10 nodes without incurring OS kernel overhead requires implementing user-space RDMA over RoCE v2 using libibverbs directly within the Zig/C execution binary[3].
The zero-copy dispatch and combine architecture interconnects physical memory between participating nodes through direct hardware channels. On the source GB10 node, the Blackwell Tensor Cores write output activations directly into pinned circular transfer buffers residing in LPDDR5X unified memory over the coherent NVLink-C2C bus[28]. The local ConnectX-7 network adapter accesses this buffer via direct DMA and dispatches data across the 400 Gbps RoCE v2 fabric using one-sided RDMA WRITE with immediate data[4]. At the target GB10 node, the receiving ConnectX-7 adapter deposits the activation payload directly into a pre-registered inbound ring buffer in LPDDR5X memory and generates an immediate completion entry, triggering expert compute kernels without CPU intervention[4].
The bare-metal network engine bypasses standard sockets, MPI, and NCCL, interacting directly with the Mellanox ConnectX-7 subsystem via userspace driver interfaces (mlx5)[31]. The software architecture maintains a persistent pool of Reliable Connection (RC) Queue Pairs (QPs) interconnecting all N nodes in the swarm:
The GB10 unified LPDDR5X memory architecture allows memory allocations to be mapped identically into CPU and GPU virtual address spaces over NVLink-C2C[28]. Pinned circular transfer buffers are registered with the ConnectX-7 HCA via ibv_reg_mr() using access flags IBV_ACCESS_LOCAL_WRITE | IBV_ACCESS_REMOTE_WRITE. Because memory is physically coherent across the Grace CPU and Blackwell GPU, zero-copy semantics apply: the Blackwell Tensor Cores write output activations directly into the registered LPDDR5X buffer, and the ConnectX-7 pulls data directly across the NVLink-C2C interconnect without intermediate

host-staging copies[3].
Work Queue Elements (WQEs) are constructed directly in user space using templated descriptors[4]. For sub-microsecond dispatch latency, the engine employs InfiniBand GPUDirect Async (IBGDA), allowing GPU CUDA kernels or high-priority Zig threads to ring the NIC doorbell register directly via Memory-Mapped I/O (MMIO) writes[3].
Dispatches utilize one-sided RDMA WRITE with an immediate 32-bit integer (IBV_WR_RDMA_WRITE_WITH_IMM)[4]. The immediate payload encodes the metadata:

- Bits 0–9: Source Token Batch Index (≤ 1024)
- Bits 10–17: Destination Expert Identifier ($E \leq 256$)
- Bits 18–25: Lookahead Routing Sequence Tag
- Bits 26–31: Routing Scale Factor (Quantized FP6/FP8 weight)

At the target node, the ConnectX-7 hardware writes the token activation directly into the pre-registered circular slot and places a Completion Queue Element (CQE) onto the completion ring[4]. The receiving GB10 core tracks inbound token arrival using an atomic ImmCounter variable, eliminating polling across the network payload itself and guaranteeing that expert compute kernels launch immediately upon packet arrival[4].

Wire Format for Token Dispatch and Combine

The engine standardizes on an aligned, zero-copy packet format optimized for NVFP4/FP8 mixed execution.

Offset (Bytes)	Field Name	Data Type	Microarchitectural Description
0x00 - 0x07	Global Token ID	uint64_t	Global sequence index and batch tracking tag
0x08 - 0x0B	Source Node Rank	uint32_t	Originating node identifier (0 to $N -$)
0x0C - 0x0F	Layer & Expert Index	uint16_t x 2	Target Layer (l) and Target Expert index (e)
0x10 - 0x13	Routing Scale Weight	float32	Top- k gating

			scalar value ($\alpha_{t,e}$)[24]
0x14 - 0x1F	Reserved Alignment	uint8_t[12]	32-byte header boundary padding
0x20 - 0x1C1F	Activation Payload	fp8_e4m3[7168]	Token hidden state vector ($7,168$ bytes)[8]
0x1C20 - 0x1CEF	Block Scaling Factors	fp8_e8m0[224]	FP4/FP8 per-tile scale factors (32:1 block ratio)[34]
0x1CF0 - 0x1CFF	Tail CRC / Epoch Stamp	uint32_t x 4	Hardware fence synchronization and integrity check

The total packet footprint is exactly 7,424 bytes, aligning to 128-byte cache line boundaries. During the MoE Combine phase, the destination node computes the expert FFN output, multiplies the resulting activation by the routing scale weight $\alpha_{t,e}$, and performs an identical one-sided RDMA WRITE back to the originating node's combine accumulation buffer, where partial results are accumulated via a fused vector add kernel[4].

Network Bandwidth Saturation and Latency Math

To evaluate network scaling, saturation metrics are derived for a 256-expert model processing a distributed batch of 4,096 tokens across clusters of $N \in \{16, 32, 64\}$ GB10 nodes. Each node is equipped with a single-port ConnectX-7 adapter operating at 400 Gbps (effective unidirectional line rate $B_{link} = 50.0 \text{ GB/s}$)[26].

Let B denote the total batch size across the cluster ($B = 4096$ tokens), N denote the number of participating GB10 nodes, k denote the number of active experts routed per token, d_{model} denote the hidden dimension ($d_{model} \in \{4096, 7168\}$), and S_{token} denote the wire footprint per dispatched token activation ($S_{token} = d_{model} \times 1 \text{ byte}$ for FP8, and $d_{model} \times 2 \text{ bytes}$ for BF16, excluding header overhead)[8].

Under uniform expert routing, the probability that a selected expert resides on a remote node is:

$$P_{remote} = \frac{N-1}{N}$$

Each node initially holds $B_{local} = \frac{B}{N}$ tokens. The total number of remote expert dispatches originated by a single node is:

$$D_{local} = B_{local} \times k \times P_{remote} = \left(\frac{B}{N}\right) \times k \times \left(\frac{N-1}{N}\right)$$

The total data transmitted by each node during the dispatch phase is:

$$M_{dispatch} = D_{local} \times S_{token} \text{ bytes}$$

The minimum theoretical network transmission time across the ConnectX-7 link is:

$$T_{trans} = \frac{M_{dispatch}}{B_{link}}$$

The calculations below detail communication volumes and transfer times across varying topologies, precisions, and routing configurations for a cluster batch of $B = 4,096$ tokens.

Nodes (N)	Top-k	Hidden Dim (d)	Precision	Token Size (Stok)	Outbound Data (Mdisp)	Min Transfer Time (Ttrans)	Network Incast Per Node
16	2	4,096	FP8	4,096 B	1.966 MB	$39.32\ \mu s$	1.966 MB
32	2	4,096	FP8	4,096 B	1.016 MB	$20.32\ \mu s$	1.016 MB
64	2	4,096	FP8	4,096 B	0.516 MB	$10.32\ \mu s$	0.516 MB
16	8	4,096	FP8	4,096 B	7.864 MB	$157.29\ \mu$	7.864 MB
32	8	4,096	FP8	4,096 B	4.063 MB	$81.26\ \mu s$	4.063 MB

64	8	4,096	FP8	4,096 B	2.064 MB	$41.29\ \mu s$	2.064 MB
16	2	7,168	FP8	7,168 B	3.441 MB	$68.81\ \mu s$	3.441 MB
32	2	7,168	FP8	7,168 B	35.55 μs	$35.55\ \mu s$	1.778 MB
64	2	7,168	FP8	7,168 B	0.903 MB	$18.06\ \mu s$	0.903 MB
16	8	7,168	FP8	7,168 B	13.763 MB	$275.25\ \mu$	13.763 MB
32	8	7,168	FP8	7,168 B	7.111 MB	$142.21\ \mu$	7.111 MB
64	8	7,168	FP8	7,168 B	3.613 MB	$72.25\ \mu s$	3.613 MB
16	8	7,168	BF16	14,336 B	27.525 MB	$550.50\ \mu$	27.525 MB
32	8	7,168	BF16	14,336 B	14.221 MB	$284.43\ \mu$	14.221 MB
64	8	7,168	BF16	14,336 B	7.225 MB	$144.51\ \mu$	7.225 MB

Total transfer cycles encompass both dispatch and combine phases; total communication latency equals $2 \times T_{trans}$ plus network switch flight time ($\approx 1.2\ \mu s$ per hop)[27]. When evaluating an alternate scaling regime where each individual node sustains a local batch of 4,096 tokens (scaling aggregate cluster batch to $4096 \times N$), individual node outbound traffic scales to $D_{local} = 4096 \times k \times \frac{N-1}{N}$ tokens[27]. In this saturation regime for $d_{model} = 7168$, $k = 8$, FP8 dispatch requires transmitting 220.2 MB ($N = 16$), 227.5 MB ($N = 32$), and 231.2 MB ($N = 64$), consuming $4.40\ \text{ms}$, $4.55\ \text{ms}$, and $4.62\ \text{ms}$

of network wire time per layer, respectively[27]. This proves that fine-grained speculative dispatch is essential to interleave compute and networking before buffers saturate.

Asynchronous NVMe Direct I/O and Hardware Decompression Subsystems

Direct I/O Yields via io_uring and O_DIRECT

To execute multi-trillion parameter models without hosting all weights in memory, non-resident experts must be streamed dynamically from high-speed storage[1]. The NVIDIA GB10 system integrates PCIe Gen 5 x4 M.2 NVMe storage interfaces[28]. Standard POSIX read() or memory-mapped (mmap) interfaces introduce severe performance degradation: page-fault exceptions block execution threads, kernel page cache management consumes host CPU cycles, and TLB shootdowns stall memory buses[11].

The engine implements a pure-C asynchronous direct I/O engine leveraging Linux io_uring[30]. The architecture applies O_DIRECT to completely bypass the Linux kernel page cache, eliminating double-buffering between kernel pages and user-space memory[38]. It configures IORING_SETUP_SQPOLL to spawn a dedicated kernel polling thread that continuously scans the submission queue ring, enabling submission of read requests without executing the io_uring_enter() system call[30]. Furthermore, it uses io_uring_register_buffers() to pre-register and pin application memory buffers with the OS kernel at startup, allowing NVMe Direct Memory Access (DMA) controllers to transfer data straight into physical LPDDR5X memory addresses without run-time page translation overhead[30].

Empirical measurements on PCIe Gen 5 x4 enterprise NVMe drives demonstrate sustained sequential read throughput of $14.0\ \mathrm{GB/s}$ under pure io_uring direct I/O with 128 KB block alignments and a queue depth of 64[27].

NVIDIA GB10 Hardware Decompression Engine and nvCOMP Integration

The NVIDIA Grace Blackwell architecture introduces an on-die Hardware Decompression Engine (DE) supported via the nvCOMP library[41]. The Blackwell DE provides native hardware acceleration for lossless algorithms including Bitcomp, Snappy, Deflate, GDeflate, and LZ4, achieving uncompressed output throughput up to $539\ \mathrm{GB/s}$ to $600\ \mathrm{GB/s}$ with latency under $1.3\ \mathrm{ms}$ for 100 MB workloads[41].

However, microarchitectural analysis of the GB10 chip reveals critical hardware implementation boundaries: High-end server Blackwell dies (B200, GB200) incorporate dedicated, autonomous hardware DE logic directly adjacent to the memory crossbar[43]. In contrast, on the GB10 (GB20B die, 48 SMs), nvCOMP pipelines decompression routines either through dedicated DE blocks or through optimized warp-level SM CUDA decompression kernels, depending on buffer registration and memory addressing[29]. As documented in the nvCOMP

Blackwell runtime specifications, the hardware DE imposes strict memory allocation rules: input and output buffers larger than 4 MB trigger fallback to SM-based CUDA decompression if they are not allocated via cudaMallocFromPoolAsync or cuMemCreate with continuous physical backing[43].

To maximize I/O throughput, NVFP4 model weights are stored on NVMe disk in a pre-compressed format using Bitcomp or GDeflate[42]. NVFP4 tensors exhibit non-uniform exponent clustering and structured mantissa distributions, enabling lossless compression algorithms to achieve a measured compression ratio of approximately $1.50\times$ without altering underlying floating-point values[27].

The weight streaming pipeline couples the PCIe Gen 5 NVMe drive directly to GPU execution memory through a multi-stage direct dataflow. The io_uring worker thread issues an asynchronous DMA read over PCIe Gen 5 x4, streaming the 16.51 MB compressed Bitcomp/GDeflate chunk from disk into a pre-registered host-pinned staging ring buffer in LPDDR5X memory at 14.0 GB/s, consuming 1.179 ms[27].

Immediately upon DMA packet arrival, the hardware Decompression Engine (or SM-assisted nvCOMP stream) uncompresses the 16.51 MB payload directly into the target Blackwell execution memory or Tensor Memory (TMEM) at 539.0 GB/s, requiring 0.046 ms[27].

The total retrieval latency is 1.225 ms, representing a 30.7% acceleration over reading raw 24.77 MB NVFP4 tensors (1.769 ms) and elevating effective storage bus yield to 20.2 GB/s[27]. Once decompressed, the expert weights are immediately accessible for warp-level tcgen05.mma matrix operations[45].

Tiered Memory Layout: Pinned LPDDR5X vs. NVMe Staging

The GB10 Superchip provides 128 GB of unified LPDDR5X memory shared coherently between the 20-core Grace CPU and the Blackwell GPU across NVLink-C2C[28]. To prevent out-of-memory (OOM) failures and bandwidth starvation, the memory space is partitioned into rigid, non-overlapping pools[14].

Memory Allocation Pool	Resident Footprint	Storage Format & Functional Allocation
Dense Transformer Backbone	12.0 GB	FP8 / NVFP4 Embeddings, Attention Projections, and LayerNorms[41]
Speculative Lookahead Router	1.5 GB	FP8 Cascaded Predictor Weights for Horizons $H \in \{1, 2, 3\}$ [5]

Pinned Hot Experts	68.0 GB	Pinned Resident Working Set: ~2,745 Fine-Grained NVFP4 Experts[12]
Sliding Window Active KV Cache	20.0 GB	Attention Sink Tokens (Initial 128) + Recent Local Sequence Tokens[13]
Dynamic Swapping Staging Buffers	8.0 GB	io_uring DMA Pinned Pages & nvCOMP Hardware DE Staging Buffers[30]
RDMA Outbound / Inbound Rings	6.0 GB	ConnectX-7 Registered Zero-Copy Circular Buffers (RoCE v2)[4]
System Overhead & OS Runtime	12.5 GB	Linux Kernel, Lumabri Household Daemon, and Zig Runtime Arenas[10]

Total allocated memory across all subsystems equals 128.0 GB. For long-context inference, the KV cache follows the Strata tiered architecture: attention sink tokens (the initial 128 sequence tokens) and the most recent 8,192 tokens remain pinned in LPDDR5X unified memory[14]. Distant intermediate context tokens are asynchronously written to NVMe via io_uring and paged back into memory only when speculative lookahead attention heads signal multi-hop retrieval requirements[13].

Microarchitectural Kernel Optimization and Blackwell Tensor Core Execution

SM120 Execution Paradigm: Fifth-Generation Tensor Cores and tcgen05

The NVIDIA GB10 Blackwell GPU architecture (SM120 / GB20B) fundamentally breaks from the warp-group-synchronous paradigm of the Hopper architecture[29]. Hopper's wgmma instruction required four warps (128 threads) to synchronize in lock-step to issue tensor operations[46]. Blackwell eliminates this barrier by introducing the tcgen05 instruction family in PTX ISA 8.7+, enabling independent warp-level (32 threads) and single-thread tensor core dispatch[45]:
Tensor operations are driven by tcgen05.mma, specifying matrix shape, numeric kind, and operand origins[45]. For NVFP4 computation, the instruction targets .kind::mxf4nvf4.block_scale, executing mixed-precision FP4 matrix multiplication with hardware-decoded 8-bit scale factors (E8M0/E4M3) natively without manual dequantization[34].

Blackwell introduces Tensor Memory (TMEM), an on-chip, SRAM-backed storage tier coupled directly to the Tensor Cores[41]. TMEM exists outside the traditional register file and shared memory (SMEM) hierarchy, bypassing the 99 KiB SMEM capacity cliff that limited Hopper tile sizes[46].
Data movement into TMEM is governed via tcgen05.cp, which streams data asynchronously from global LPDDR5X memory or L2 cache directly into TMEM tiles[41]. Data transfer between TMEM and registers or SMEM is executed using tcgen05.ld and tcgen05.st[41].
Software-managed allocation and release of TMEM address space are controlled using tcgen05.alloc and tcgen05.dealloc[46].

Model Bandwidth Utilization (MBU) in Sparse MoE Execution

In standard dense LLM inference, batched matrix operations achieve high Model Bandwidth Utilization (MBU) because large weight matrices are reused across all sequence tokens[2]. In sparse MoE decode execution, however, token routing is sparse and bursty[2]. If an expert receives only a single token or a small handful of tokens in a generation step, execution degenerates into memory-bound vector-matrix multiplication[2].
To maximize MBU, the engine incorporates a persistent Fused GroupedGEMM kernel written in low-level PTX assembly. Within each SM, execution is double-buffered across ping-pong TMEM tiles[41]. While the Tensor Cores compute the GEMM for token chunk i using weights residing in TMEM Tile 0, an asynchronous tcgen05.cp instruction fills TMEM Tile 1 with weights for token chunk $i+1$ directly from the L2 cache[41].
Simultaneously, tokens dispatched from the RDMA network destined for the same local expert are grouped into contiguous execution buckets within LPDDR5X[35]. Rather than issuing independent kernel launches for each expert, a single persistent GroupedGEMM kernel processes all local experts, eliminating kernel launch overhead and maximizing Tensor Core occupancy[35].

Comprehensive Hardware Bottleneck Analysis and Mathematical Derivations

Analytical Hardware Specifications

Calculations are evaluated against the verified physical specifications of the NVIDIA GB10 Grace Blackwell Superchip (DGX Spark architecture)[28]:

- CPU: 20-Core Arm Architecture (10 Cortex-X925 + 10 Cortex-A725)[28]
- GPU: Blackwell Generation (GB20B, 48 SMs, 6,144 CUDA Cores, 192 5th-Gen Tensor Cores)[28]
- Memory Subsystem: 128 GB LPDDR5X Unified System Memory[28]
- Memory Interface: 256-bit bus operating at 8.5 Gbps effective[28]
- Peak Memory Bandwidth: $273.2\ \mathrm{GB/s}$[28]

- Peak Compute Throughput (Dense FP4): $475.4\ \text{TFLOPS}$ ($475.4 \times 10^{12}\ \text{FLOP/s}$)[29]
- Peak Compute Throughput (Sparse 2:4 FP4): $950.8\ \text{TFLOPS}$ [29]
- Storage Interface: PCIe Gen 5 x4 NVMe ($14.0\ \text{GB/s}$ sustained direct I/O read)[28]
- Network Interface: NVIDIA ConnectX-7 ($400\ \text{Gbps} = 50.0\ \text{GB/s}$ unidirectional line rate)[26]
- Hardware Decompression Engine: $539.0\ \text{GB/s}$ uncompressed output bandwidth[27]

Operational Intensity and Arithmetic Saturation

Consider a single expert with $P_{expert} = 44.04 \times 10^6$ parameters[27]. Under NVFP4 encoding (4 bits per parameter plus 8-bit block scale factors per 16 weights, yielding 0.5625 bytes/parameter), the memory footprint of one expert is[27]:

$$M_{expert} = 44,040,192 \times 0.5625\ \text{B} = 24,772,608\ \text{Bytes} \approx 24.77\ \text{MB}$$

The machine balance point (critical operational intensity) of the GB10 Superchip at peak dense FP4 compute is:

$$I_{machine} = \frac{\text{Peak Compute}}{\text{Peak Memory Bandwidth}} = \frac{475.4 \times 10^{12}\ \text{FLOP/s}}{273.2 \times 10^{9}\ \text{B/s}} = 1,740.12\ \text{FLOP/Byte}$$

When processing a batch of T_e tokens routed to this expert, the computation requires:

$$\text{FLOPs}(T_e) = 2 \times P_{expert} \times T_e = 2 \times 44,040,192 \times T_e = 88.08 \times 10^6 \times T_e\ \text{FLOPs}$$

The operational intensity of the expert execution is:

$$I(T_e) = \frac{\text{FLOPs}(T_e)}{M_{expert}} = \frac{88.08 \times 10^6 \times T_e}{24.77 \times 10^6\ \text{Bytes}} \approx 3.556 \times T_e\ \text{FLOP/Byte}$$

To saturate the Blackwell Tensor Cores and transition from memory-bandwidth-bound execution to compute-bound execution, the expert must satisfy $I(T_e) \geq I_{machine}$:

$$3.556 \times T_e \geq 1740.12 \implies T_e \geq \frac{1740.12}{3.556} \approx 489.3\ \text{tokens}$$

The mathematical relationships across token batch sizes, operational intensities, and execution times are summarized in the table below.

Tokens	Total	Operatio	Memory	Peak	Limiting	Model

Routed to Expert (Te)	FLOPs (109)	nal Intensity (FLOP/B)	Load Time (tmem)	Compute Time (tcomp)	Bottleneck	Bandwidth Utilization (MBU)
1	0.088	3.56	$90.68\ \mu s$	$0.19\ \mu s$	LPDDR5X Bandwidth	99.8% (Memory Bound)
8	0.705	28.45	$90.68\ \mu s$	$1.48\ \mu s$	LPDDR5X Bandwidth	98.4% (Memory Bound)
32	2.819	113.79	$90.68\ \mu s$	$5.93\ \mu s$	LPDDR5X Bandwidth	93.9% (Memory Bound)
64	5.637	227.58	$90.68\ \mu s$	$11.86\ \mu s$	LPDDR5X Bandwidth	88.4% (Memory Bound)
128	11.274	455.11	$90.68\ \mu s$	$23.71\ \mu s$	LPDDR5X Bandwidth	79.3% (Memory Bound)
256	22.549	910.22	$90.68\ \mu s$	$47.43\ \mu s$	LPDDR5X Bandwidth	65.6% (Memory Bound)
489	43.071	1,739.00	$90.68\ \mu s$	$90.60\ \mu s$	Equal Balance	50.0% (Inflection Point)
1,024	90.194	3,641.34	$90.68\ \mu s$	$189.72\ \mu s$	FP4 Tensor Compute	24.3% (Compute Bound)

During interactive autoregressive token generation (decode phase), tokens per expert typically

range from $T_e = 1$ to $T_e = 64$ across a distributed cluster[2]. Consequently, single-token MoE decode execution on GB10 is strictly memory bandwidth bound, requiring exactly $90.68\ \mu s$ to stream the expert from LPDDR5X regardless of whether 1 or 32 tokens are processed[27].

PCIe Gen 5 Weight Chunk Swapping Latency Math

When an expert must be retrieved from NVMe storage, latency is evaluated across raw uncompressed reads versus compressed streaming through the Blackwell Hardware Decompression Engine (DE)[27].

Target Weight Chunk	Format / Data Size	Compression Ratio	On-Disk Footprint	PCIe 5.0 Read Time (tio)	Blackwell DE Latency (tdecomp)	Total Retrieval Latency	Effective I/O Throughput
Single Expert	NVFP4 Raw (24.77 MB)	$1.0\times$	24.77 MB	1.769 ms	0.000 ms	1.769 ms	14.00 GB/s
Single Expert	NVFP4 Bitcomp	$1.5\times$	16.51 MB	1.179 ms	0.046 ms	1.225 ms	20.22 GB/s
5.0 GB Weight Chunk	NVFP4 Raw (5.00 GB)	$1.0\times$	5.00 GB	357.14 ms	0.000 ms	357.14 ms	14.00 GB/s
5.0 GB Weight Chunk	NVFP4 Bitcomp	$1.5\times$	3.33 GB	238.10 ms	9.28 ms	247.38 ms	20.21 GB/s
10.0 GB Weight Chunk	NVFP4 Raw (10.00 GB)	$1.0\times$	10.00 GB	714.29 ms	0.000 ms	714.29 ms	14.00 GB/s

10.0 GB Weight Chunk	NVFP4 Bitcomp	$1.5\times$	6.67 GB	476.19 ms	18.55 ms	494.74 ms	20.21 GB/s

The mathematical modeling proves that loading multi-gigabyte weight slices (e.g., 5 GB to 10 GB) incurs a 247 ms to 495 ms penalty[27]. Thus, dynamic weight swapping cannot be performed on a layer-by-layer basis during decode without stalling token generation. Dynamic NVMe streaming is feasible only at the granularity of individual fine-grained experts (1.225 ms per expert) governed by lookahead routing horizons[27].

System Architecture Blueprint: Threading Model, Memory Rings, and Execution Pipeline

Layered Software Stack

The engine unifies Zig-based bare-metal scheduling, pure-C distributed memory management, and low-level Blackwell assembly kernels into a cohesive software architecture[10].

Layer Index	Architectural Subsystem	Implementation Technologies	Microarchitectural Responsibilities
Layer 4	High-Level Orchestration & Swarm Topology	Lumabri Household Tracker (Pure-C Daemon)	Node Health, UDP/TCP 47300 Consensus, Replica Allocation[10]
Layer 3	Speculative Execution & Dispatch Pipeline	TensorFold Engine (Zig Core Runtime)	Speculative Lookahead Loop, Arena Allocation, Token Batching[15]
Layer 2	Communication & Storage Subsystems	libibverbs (RoCE v2) + io_uring + nvCOMP	RDMA WRITE_IMM, SQPOLL Direct I/O, Hardware DE Streams[4]
Layer 1	Microarchitectural Execution Kernels	Strata PTX Assembly Kernels	tcgen05.mma, TMEM Ping-Pong

		(sm_120)	Allocation, NVLink-C2C Coherence[41]

Multi-Threaded Execution Architecture and Memory Rings

To eliminate pipeline bubbles, execution is decoupled across four asynchronous OS threads pinned to dedicated Grace CPU cores, communicating via wait-free, lock-free ring buffers[54].
The Speculative Router Thread executes on Grace Cortex-X925 Core 1, continuously evaluating the cascaded predictor across lookahead horizons $H \in \{1, 2, 3\}$ and emitting prefetch descriptors into the Prefetch Command Ring Buffer[5].
The NVMe Streaming Worker Thread operates on Cortex-X925 Core 2, consuming prefetch commands, submitting asynchronous DMA requests via io_uring direct I/O, managing nvCOMP decompression streams, and updating the atomic Expert Ready State Table[28].
The RDMA Communications Proxy runs on Cortex-A725 Core 3, constructing zero-copy WQEs, posting MMIO doorbells to the ConnectX-7 HCA, and monitoring completion events for incoming token packets[4].
The GPU Dispatch and Compute Engine is orchestrated by Cortex-A725 Core 4 in coordination with the 48 Blackwell SMs, fusing inbound token activations into persistent GroupedGEMM kernels, executing matrix computations, and streaming combined outputs into outbound RDMA rings[28].
The execution pipeline enforces synchronization contracts using memory-mapped ring buffers structured as follows:
The Prefetch Command Ring Buffer is a single-producer, single-consumer (SPSC) circular queue connecting the Speculative Router Thread to the NVMe Streaming Worker. Each entry contains a 64-byte descriptor specifying expert_id, layer_id, destination_lpddr5x_addr, and urgency_deadline_us.
The Expert Ready State Table consists of an atomic bitmask table (_Atomic uint64_t expert_residency[L][E/64]). When an expert is decompressed and resident in LPDDR5X, the NVMe worker sets the corresponding bit using atomic_fetch_or_explicit(..., memory_order_release). The compute engine validates residency via zero-cost atomic load before kernel launch.
The Token Activation Inbound and Outbound Rings are pinned circular buffers mapped directly to ConnectX-7 memory keys ($rkey$). Each slot accommodates a 7,424-byte packet. When a packet arrives, the NIC places the immediate data into the completion ring, incrementing an atomic counter (ImmCounter). The GPU kernel checks this counter using PTX ld.acquire.sys instructions to begin processing without waiting for CPU wakeups[4].

Prefetch Horizon Pipeline Execution Flow

The chronological progression of execution through layers l, $l+$, and $l+2$ illustrates how speculative lookahead completely hides communication and storage overhead.

Relative Timeline	Hardware Subsystem	Operational Phase & Execution Action
$T =$	GPU SM120 Compute	Initiates Layer l Self-Attention matrix multiplication on active batch tokens[29]
$T =$	Grace CPU Core 1	Evaluates Speculative Lookahead Router for Horizons $l +$ and $l + 2$ [5]
$T =$	Grace CPU Core 1	Completes prediction; issues $l +$ RDMA pre-negotiation and $l + 2$ NVMe prefetch commands[5]
$T =$	Grace CPU Core 2	Submits io_uring direct I/O read for Layer $l + 2$ non-resident experts (16.51 MB)[27]
$T =$	GPU SM120 Compute	Completes Layer l Self-Attention; initiates RDMA token dispatch via IBGDA[3]
$T =$	ConnectX-7 Fabric	Completes Layer l All-to-All token dispatch across 32 nodes ($7.11\ \mathrm{ME}$)[26]
$T =$	GPU SM120 Compute	Launches Layer l GroupedGEMM across local resident experts[35]
$T =$	GPU SM120 Compute	Completes Layer l MoE

		GEMM; dispatches Layer l Combine activations over RDMA[4]
$T =$	ConnectX-7 Fabric	Completes Layer l Combine exchange; GPU initiates Layer $l +$ Self-Attention[4]
$T =$	NVMe DMA Interface	Completes io_uring DMA transfer of compressed Layer $l + 2$ expert weights[27]
$T =$	Hardware DE Engine	Decompresses Layer $l + 2$ weights into LPDDR5X execution buffers ($46\ \mu s$)[27]
$T =$	Ready State Table	Marks Layer $l + 2$ missing experts as RESIDENT in atomic bitmask[54]
$T =$	GPU SM120 Compute	Completes Layer $l +$ execution; transitions into Layer $l + 2$ GEMM with zero I/O wait bubbles[27]

Critical Failure Modes and Low-Level Engineering Mitigations

Failure Mode 1: Speculative Lookahead Misprediction Penalties and Pipeline Freezing

If the speculative auxiliary router mispredicts the top-k expert set for layer $l + 2$, an un-prefetched, non-resident expert will be required during layer $l + 2$'s actual execution[5]. Because transferring an uncompressed expert from NVMe consumes $1.769\ \text{ms}$ (or

$1.225\ \text{ms}$ compressed), a cache miss directly blocks the GPU execution pipeline[27]. If a batch encounters a 10% misprediction rate across 60 layers, the engine suffers an accumulated penalty of:

$$60 \times 0.10 \times 1.225\ \text{ms} = 7.35\ \text{ms of idle GPU stalling per token generation step}$$

This reduces token generation throughput from an interactive 35 tokens/s down to less than 8 tokens/s, undermining real-time deployment[33].

To resolve misprediction stalls, the engine implements a multi-tiered resilience pipeline. First, the auxiliary router over-provisions speculative selection by choosing $k + \Delta k = 10$ candidates rather than strictly $k = 8$ [5]. The top 8 are scheduled for primary execution, while candidates 9 and 10 are pre-fetched into a transient ring buffer if they are non-resident, absorbing over 94% of boundary mispredictions[5].

Second, the engine incorporates skip-weight gating derived from Strata[48]. If a token routes to a non-resident expert that was omitted from the prefetch horizon, the engine evaluates its routing weight $\alpha_{t,e}$. If $\alpha_{t,e}$ falls below a skipping threshold τ (e.g., $\tau \leq 0.05$), the engine prunes that expert from the token's active set and renormalizes the weights across the remaining resident experts[5]:

$$\hat{\alpha}_{t,j} = \frac{\alpha_{t,j}}{\sum_{m \in \mathcal{E}_{t,resident}} \alpha_{t,m}}$$

Empirical evaluations demonstrate that skipping low-weight expert misses produces negligible perplexity drift ($\Delta\text{PPL} < 0.05\%$) while eliminating execution stalls[41].

Third, if $\alpha_{t,e} > \tau$ and the expert is missing locally, the engine bypasses the local NVMe drive entirely. Instead, the Lumabri tracker queries the swarm for a peer holding that expert pinned in LPDDR5X, pulling the expert across the 400 Gbps ConnectX-7 fabric via one-sided RDMA READ in $0.495\ \text{ms}$, reducing the stall penalty by 59.6% compared to local PCIe Gen 5 SSD reads[10].

Failure Mode 2: RDMA All-to-All Incast Congestion and Head-of-Line Blocking

Because expert activation is data-dependent, real-world workloads exhibit sharp routing skews where specific popular experts receive massive bursts of tokens from across the cluster[6]. In a 64-node swarm, if an expert hosted on Node 0 is selected by tokens originating across all 63 other nodes, Node 0 experiences a severe 63:1 network incast[26]. Under standard RoCE v2, this incast immediately overflows the ConnectX-7 receive packet buffers, triggering IEEE 802.1Qbb Priority Flow Control (PFC) pause frames[26]. These pause frames propagate backward upstream to the network switches, causing tree saturation, head-of-line blocking, and catastrophic

throughput collapse across unrelated traffic[26].
To prevent incast collapse, the engine implements three complementary network controls. Inbound traffic is regulated through receiver-driven credit allocation inspired by Ultra Ethernet Consortium (UEC) standards[26]. Nodes cannot transmit activations to an expert host without consuming an explicit receiver-granted credit token. The receiving node meters credit grants based on its remaining LPDDR5X ring buffer capacity, preventing hardware queue overflow and eliminating PFC pause generation[26].
Simultaneously, the Lumabri household tracker continuously monitors token arrival frequencies to compute the Load Imbalance Factor (LIF) across all experts[6]:

$$\mathrm{LIF}_e = \frac{T_e}{\frac{1}{E}\sum_{i=1}^{E} T_i}$$

When an expert's activation exceeds threshold $\mathrm{LIF}_e \geq 2.5$, the tracker triggers dynamic online replication, mirroring the expert weights to secondary nodes during idle communication phases[5]. The speculative routers update their placement routing maps, splitting token dispatches across the replica set and eliminating the network hotspot[5].
Finally, the network transport layer configures Data Center Quantized Congestion Notification (DCQCN) with aggressive Explicit Congestion Notification (ECN) marking at switch buffer thresholds of 20%, forcing sending nodes to down-throttle injection rates within sub-round-trip timescales before packet drops can occur[26].

Failure Mode 3: Unified LPDDR5X Memory Bus Contention and Bandwidth Thrashing

The NVIDIA GB10 architecture features a unified 256-bit LPDDR5X memory system delivering $273.2\ \mathrm{GB/s}$ of aggregate bandwidth shared across the entire System-on-Chip (SoC)[28]. Unlike discrete datacenter GPUs where HBM3e delivers $4,800\ \mathrm{GB/s}$ to $8,000\ \mathrm{GB/s}$ exclusively to the GPU, all GB10 subsystems contend for the same physical memory bus[33]. Blackwell Tensor Cores running FP4 GEMM require up to $273.2\ \mathrm{GB/s}$, PCIe Gen 5 NVMe direct I/O transfers consume $14.0\ \mathrm{GB/s}$, ConnectX-7 RDMA outbound/inbound transfers consume up to $50.0\ \mathrm{GB/s}$, and Grace CPU execution, operating system scheduling, and display output consume $10\ \mathrm{GB/s}$ to $20\ \mathrm{GB/s}$ [28]. If all subsystems access LPDDR5X concurrently, severe memory bus contention degrades effective bandwidth by 40% to 60%, stalling the Tensor Cores and causing execution times to deviate unpredictably from calculated theoretical limits.
The engine resolves bus contention through phased memory-bus arbitration and SRAM-tier caching. The Zig execution loop enforces strict temporal separation of bus-intensive operations. NVMe DMA write-bursts and hardware decompression passes are restricted to windows where the GPU is executing compute-bound operations (such as multi-head

self-attention Softmax or LayerNorm) or waiting on network dispatch[41]. During the execution of the memory-bound MoE GroupedGEMM, background NVMe streaming is throttled via io_uring rate limiters.
Furthermore, decompressed weights and inbound RDMA activation tiles are directed into the GPU's 24 MB L2 cache and SM Tensor Memory (TMEM) rather than being written back to main LPDDR5X memory[29]. By keeping working sets pinned in on-chip SRAM via tcgen05.cp and L2 cache allocation policies, intermediate traffic is isolated from the 256-bit physical LPDDR5X bus, preserving the full $273.2\ \mathrm{GB/s}$ bandwidth for primary weight streaming[41].

Synthesis and Strategic Outlook

The architectural integration of TensorFold, Lumabri, and Strata onto an edge-scale NVIDIA GB10 swarm establishes a feasible foundation for multi-trillion parameter MoE inference without datacenter-grade multi-GPU nodes[10].
The quantitative bottleneck analysis confirms that single-token MoE decode phases on the GB10 Superchip are fundamentally memory bandwidth bound rather than compute bound; the operational intensity (3.56 to $227.6\ \mathrm{FLOP/Byte}$) falls significantly below the machine balance point of $1,740.1\ \mathrm{FLOP/Byte}$ [27]. Consequently, high Model Bandwidth Utilization (MBU) depends entirely on the efficiency of data movement across memory tiers[2].

Subsystem Domain	Measured Physical Capacity	Serving System Operational Requirement	Engineering Feasibility Status
Speculative Lookahead Routing	1.36% Auxiliary Compute Overhead	$<$ Total Compute Budget	**Fully Feasible** (Sub-microsecond evaluation)[27]
Lookahead Prefetch Horizon Lead	$300\ \mu s$ to $1.200\ \mu s$ Lead Time	$>$ RDMA Arming & Setup	**Fully Feasible** (Hides network latency)[5]
RDMA Token Dispatch (FP8)	$72\ \mu s$ to $275\ \mu s$ Network Wire Time	$<$ Layer Execution Budget	**Fully Feasible** (Zero-copy IBGDA / RoCE v2)[4]
Compressed Expert NVMe Swap	$1.225\ \mathrm{ms}$ Retrieval per Expert	$<$ Multi-Layer Horizon	**Conditionally Feasible** (Requires Bitcomp / DE)[27]

Coarse Multi-GB Weight Swap	$247\ \text{ms}$ to $495\ \text{ms}$ Retrieval Penalty	Real-Time Interactive Decode Stream	**Strictly Infeasible** (Stalls generation loop)[27]
Unified LPDDR5X Bus Concurrency	$273.2\ \text{GB/s}$ Shared Physical Bus	Contention Avoidance Across DMA / GEMM	**Requires Phased Arbitration Scheduling**[28]

The mathematical findings dictate three fundamental deployment mandates:
Coarse-grained weight swapping (5 GB to 10 GB chunks) during active inference is mathematically unviable, incurring unacceptable latency penalties exceeding 240 ms[27]. Weight streaming from NVMe must operate exclusively at the granular scale of individual fine-grained experts ($\approx 24.8\ \text{MB}$ uncompressed, $16.5\ \text{MB}$ compressed), hidden behind speculative lookahead horizons[27].

Speculative lookahead routing ($H = 2$ to 3) provides the exact temporal window ($300\ \mu\text{s}$ to $1.2\ \text{ms}$) required to bridge ConnectX-7 RDMA dispatch and hardware-accelerated NVMe prefetching[5].
The software engine must eliminate OS virtualization layers, running a unified Zig/C binary that drives libibverbs RDMA doorbells, io_uring direct I/O rings, and Blackwell tcgen05 assembly directly against bare hardware[15].
Implementing these mitigations transforms a cluster of commodity GB10 desktop nodes into a distributed inference engine capable of serving frontier models with sub-second time-to-first-token and sustained interactive generation throughput.

Works cited

1. Colibri - Run Frontier MoE Models From Disk | AI/TLDR, https://ai-tldr.dev/tools/colibri/
2. How GLM 5.2 Runs 40B Active Parameters | MindStudio, https://www.mindstudio.ai/blog/mixture-of-experts-architecture-glm-5-2-active-parameters
3. Inside the Nebius + PyTorch DeepSeek V3 recipe: NVSHMEM and, https://nebius.com/blog/posts/inside-the-nebius-pytorch-deepseek-v3-recipe
4. RDMA Point-to-Point Communication for LLM Systems - Perplexity, https://www.perplexity.ai/hub/blog/rdma-point-to-point-communication-for-llm-systems
5. Director: Accelerating Distributed MoE Serving via Online Proactive, https://arxiv.org/html/2607.08782v1
6. MoE Load Balancing: Token Distribution & Expert Collapse, https://mbrenndoerfer.com/writing/moe-load-balancing-expert-collapse-token-distribution

7. fabric-lib: RDMA Point-to-Point Communication for LLM Systems, https://proceedings.mlsys.org/paper_files/paper/2026/file/dea9b4b6f55ae611c54065d6fc750755-Paper-Conference.pdf
8. UCCL-EP: Portable Expert-Parallel Communication - arXiv, https://arxiv.org/pdf/2512.19849
9. GPU-Initiated Networking for NCCL - arXiv, https://arxiv.org/html/2511.15076v1
10. JustVugg/lumabri: Run large AI models across your computers and, https://github.com/JustVugg/lumabri
11. Colibri Made Me Rethink How We Run LLMs — And There Might Be, https://dev.to/lhabacuc/-colibri-made-me-rethink-how-we-run-llms-and-there-might-be-a-next-step-52bj
12. Colibri: How to Run a 744-Billion-Parameter Model on Consumer, https://betterstack.com/community/guides/ai/colibri-glm/
13. Strata: Hierarchical Context Caching for Long Context Language, https://arxiv.org/html/2508.18572v1
14. Strata on an RTX 5070 Ti 16GB + 64GB RAM: real agent workloads, https://www.reddit.com/r/LocalLLM/comments/1x2vdk8/strata_on_an_rtx_5070_ti_16gb_64gb_ram_real_agent/
15. TensorFold: LLM inference tool - Open Tool Atlas, https://www.opentoolatlas.com/l/tensorfold/
16. TensorFold GitHub Explained: Exact Speculative Decoding Without, https://www.youtube.com/watch?v=excMs2gI2RM
17. GitHub - ashhart/TensorFold: LLM Inference Engine for Metal, CUDA, https://github.com/ashhart/TensorFold
18. Accelerating Distributed MoE Serving via Online Proactive Expert, https://arxiv.org/pdf/2607.08782
19. Accelerating Distributed MoE Serving via Online Proactive Expert, https://arxiv.org/abs/2607.08782
20. fabric-lib: RDMA Point-to-Point Communication for LLM Systems, https://arxiv.org/abs/2510.27656
21. From Colibrì to Lumabri: running huge MoE models as a swarm of, https://www.reddit.com/r/LocalLLM/comments/1vjq6ba/from_colibr%C3%AC_to_lumabri_running_huge_moe_models/
22. Run GLM-5.2 on a CPU with 24GB RAM: How Colibri Works, https://gnana70.medium.com/run-glm-5-2-on-a-cpu-with-24gb-ram-how-colibri-works-c2cc68ea628e
23. From Colibrì to Lumabri: running huge MoE models as a swarm of, https://www.reddit.com/r/LLMDevs/comments/1vjqa06/from_colibr%C3%AC_to_lumabri_running_huge_moe_models/
24. How do mixture-of-experts layers affect transformer models?, https://stackoverflow.blog/2024/04/04/how-do-mixture-of-experts-layers-affect-transformer-models/
25. ERMoE: Eigen-Reparameterized Mixture-of-Experts for Stable, https://openaccess.thecvf.com/content/CVPR2026/papers/Cheng_ERMoE_Eigen-Reparameterized_Mixture-of-Experts_for_Stable_Routing_and_Interpretable_Spe

cialization_CVPR_2026_paper.pdf
26. UEC vs RDMA: Complete Technical Reference + NVMe-oF, https://sampooni.github.io/uec-rdma-docs/
27. unknown_url
28. NVIDIA DGX Spark - Micro Center, https://www.microcenter.com/site/brands/nvidia-dgx-spark.aspx
29. NVIDIA GB10 Specs - GPU Database - TechPowerUp, https://www.techpowerup.com/gpu-specs/gb10.c4342
30. A systematic study of libaio, SPDK, and io_uring - Large Research, https://atlarge-research.com/pdfs/2022-systor-apis.pdf
31. DOCA Perftest - NVIDIA Networking, https://networking-docs.nvidia.com/doca/archive/3-5-0/doca-perftest
32. SigmaTrace: Library-Agnostic Observability for CPU/GPU-Initiated, https://arxiv.org/html/2610.24328v1
33. NVIDIA GB10 (DGX Spark) Price: $6,950, Specs & Buy vs Rent, https://www.spheron.network/gpu-price/gb10/
34. Blackwell SM100 GEMMs — NVIDIA CUTLASS Documentation, https://docs.nvidia.com/cutlass/4.3.1/media/docs/cpp/blackwell_functionality.html
35. mKernel/README.md at main - uccl-project - GitHub, https://github.com/uccl-project/mKernel/blob/main/README.md
36. Optimizing Communication for Mixture-of-Experts Training with, https://developer.nvidia.com/blog/optimizing-communication-for-mixture-of-experts-training-with-hybrid-expert-parallel/
37. We Just Hit the Local LLM Tipping Point (Colibri) - YouTube, https://www.youtube.com/watch?v=19xCOJxWU0A
38. I/O Bandwidth - Linux Kernel Internals, https://kernel-internals.org/io/io-bandwidth/
39. io_uring for High-Performance DBMSs: When and How to Use It, https://arxiv.org/html/2512.04859v1
40. Managing SSD Storage in Database Systems - mediaTUM, https://mediatum.ub.tum.de/doc/1836218/document.pdf
41. NVIDIA GB10 Grace Blackwell Architecture - Emergent Mind, https://www.emergentmind.com/topics/nvidia-gb10-grace-blackwell
42. NVIDIA nvCOMP - NVIDIA Developer, https://developer.nvidia.com/nvcomp
43. Speeding Up Data Decompression with nvCOMP and the NVIDIA, https://developer.nvidia.com/blog/speeding-up-data-decompression-with-nvcomp-and-the-nvidia-blackwell-decompression-engine/
44. cuPhoton/docs/components/xdr.md at main - GitHub, https://github.com/NVIDIA/cuPhoton/blob/main/docs/components/xdr.md
45. and Source-Level Auditof INT8 Availability on NVIDIA Blackwell Ultra, https://arxiv.org/html/2608.11693v1
46. Microbenchmarking NVIDIA's Blackwell Architecture - arXiv, https://arxiv.org/html/2512.02189v3
47. NVIDIA DGX Spark: AI Supercomputer on Your Desk, https://www.nvidia.com/en-us/products/workstations/dgx-spark/

48. Cudecnik/Nemotron-3.5-Super-120B-A12B-NVFP4-Strata-Blackwell, https://huggingface.co/Cudecnik/Nemotron-3.5-Super-120B-A12B-NVFP4-Strata-Blackwell
49. Hardware Overview — DGX Spark User Guide, https://docs.nvidia.com/dgx/dgx-spark/hardware.html
50. The Longest Nvidia PTX Instruction | Ash's Blog, https://ashvardanian.com/posts/longest-ptx-instruction/
51. Blackwell GPU Wiki - GitHub Pages, https://0xsero.github.io/blackwell-gpu-wiki/
52. RTX PRO 6000 Blackwell vs NVIDIA GB10 Grace Benchmark, https://petronellatech.com/blog/rtx-pro-6000-blackwell-vs-nvidia-gb10-grace-benchmark/
53. Democratizing Large-Scale Mixture-of-Experts Training with NVIDIA, https://developer.nvidia.com/blog/accelerating-large-scale-mixture-of-experts-training-in-pytorch/
54. fabric-lib: RDMA Point-to-Point Communication for LLM Systems, https://www.alphaxiv.org/abs/2510.27656